

Problem: The Order State Machine

Context

Gametime is a mobile-first ticket marketplace built around last-minute purchases for sports, concerts, and theater events across 60+ cities in the US and Canada. Our checkout flow is optimized for speed—fans can go from browsing to having tickets in hand in under a minute.

Behind that seamless experience is a checkout backend where an order moves through several states: it's created, payment is authorized, and then it's completed. Each step can fail, and how you recover depends on where the failure happens.

The Problem

Order state transitions aren't simple. A payment decline is straightforward—reject the order, no cleanup needed. But a completion failure after payment has already been authorized requires a void, and if that void also fails, you're in a partial failure state that can't be silently swallowed.

Getting this wrong means fans are charged for tickets they never receive, or orders are stuck in limbo with no clear resolution path. The checkout backend needs to enforce valid transitions, handle each failure mode with the appropriate recovery, and surface partial failures for manual resolution when automation can't recover cleanly.

Your Challenge

Build a small service that models an order state machine with stage-dependent failure recovery.

The happy path:

initialized → payment_authorized → complete

What can go wrong:

- Payment is declined → reject the order. No cleanup needed.
- Completion fails after payment was authorized → void the payment, then mark the order as cancelled.
- Completion fails and the void also fails → move to needs_attention for manual resolution. Don't silently swallow the error.

Your service should:

1. Model the state machine. Define states, enforce valid transitions, and record state history with timestamps.
2. Handle failure differently depending on stage. A payment decline just rejects. A completion failure after authorization triggers a void. Same concept (something failed), different recovery.
3. Handle partial failures. If the void fails, surface it—don't pretend the order is cleanly cancelled.
4. Expose a small API for creating an order, advancing its state, and querying its current state + history.

Constraints and Notes

- Any language
- Stub payment as an interface
- Include tests: happy path, payment decline, completion failure with successful void, completion failure with failed void

What We're Looking For

- A working prototype that demonstrates your approach
- Your reasoning about the problem space and why you chose this solution
- Clean, well-documented code in a repo we can review
- A brief explanation of how you'd improve or extend this given more time

Time Expectation

We expect this to take roughly 3 hours using modern development tools. Don't over-engineer it—we want to see how you think and build, not a production-ready system.

Submission

Create a GitHub repo with your solution. Include a README that explains:

- What you built and why
- How to run it
- What tradeoffs you made
- What you'd do differently with more time